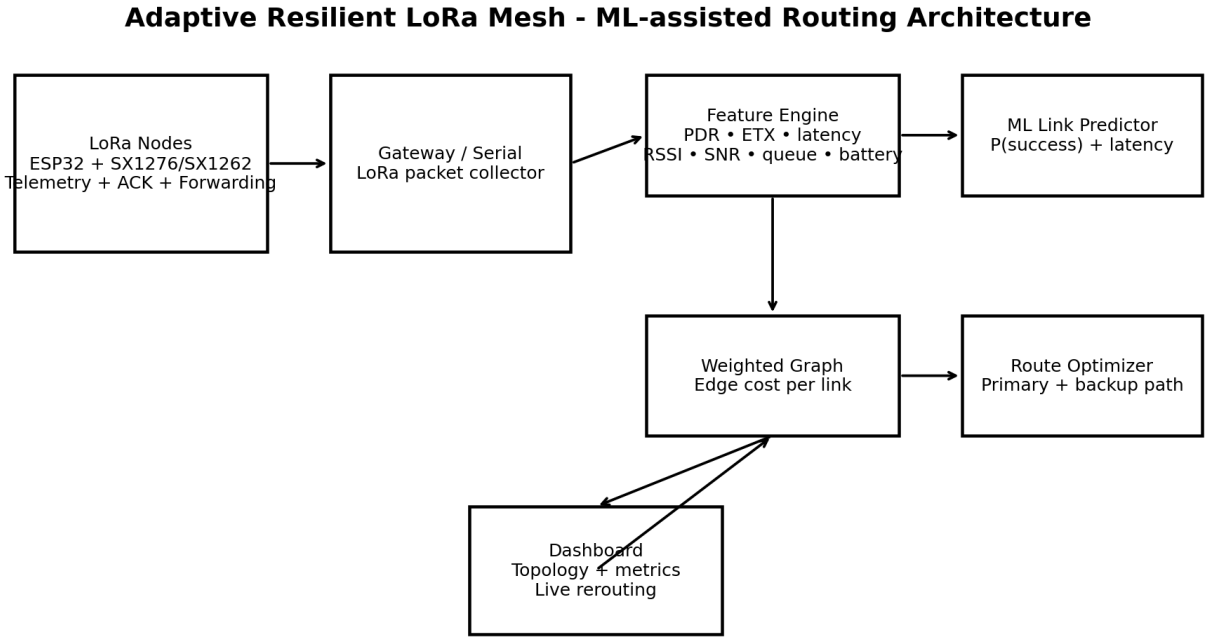
ADAPTIVE LORA MESH
ML-ASSISTED ROUTE ANALYSIS

A practical project plan for building the ML + live routing demo by tomorrow



Prepared 18 August 2026

1. Executive recommendation: what to build tomorrow

Do not spend tomorrow trying to build a full physical LoRa mesh or a full OMNeT++ model. For the first demonstration, build a software-only network digital prototype in Python. The fastest credible stack is: **Python + scikit-learn + NetworkX + Streamlit + synthetic link data,** then validate the feature choices against public LoRa datasets. The live dashboard lets you deliberately damage a link (for example, lower SNR/PDR or raise latency/retries), watch its predicted link probability fall, and show the route move to the backup path.

Need	Tomorrow choice	Why
ML model	Random Forest classifier	Fast for tabular data; easy probability output; no GPU needed.
Routing	NetworkX shortest path over ML-derived edge costs	Separates prediction from routing and is easy to visualize.
Live demo	Streamlit sliders	User can change a metric during the demo and immediately see the route change.
Data	Public LoRa data + synthetic augmentation	Real measurements support credibility; synthetic data fills missing route-level labels for the demo.
Full radio simulator	Do later, not tonight	FLoRa/OMNeT++ and ns-3 are valuable but setup and modeling work are substantially larger.

Core design rule: ML predicts link condition; the routing algorithm selects the path; the packet protocol guarantees correctness through integrity checks, sequence numbers and acknowledgements. Do not make the ML model directly output the route.

2. Problem statement and project objective

The project is an adaptive, resilient long-range mesh communication prototype. Each node reports measurements of its radio link and its own operational state. A central computer builds a live network graph, estimates which links are likely to succeed, and selects a primary and backup route. When a link degrades, the route is recomputed and the new next-hop information is pushed to the nodes in the prototype architecture.

Layer	Question it answers	Examples
Measurement	What is happening on each link/node?	RSSI, SNR, PDR, retries, latency, queue, battery, temperature
ML	How healthy is this link likely to be?	P(success) and optionally predicted latency
Routing	Which path should be used?	Primary path, backup path, route hysteresis
Protocol	Did the message arrive correctly?	CRC/integrity, sequence number, ACK, duplicate detection, timeout
Visualization	Can an evaluator see the decision?	Topology, link probabilities, route change, metrics

3. Recommended data sources

There is no single public dataset that contains every feature in your screenshot (RSSI, SNR, PDR, retries, latency, battery, CPU, queue and temperature) for a multi-hop mesh. The best approach is to use one or two real LoRa datasets for radio/environmental realism and generate the missing node/mesh features synthetically for a controlled demo.

Source	URL	What you can use
1. ChirpBox long-term LoRa	https://zenodo.org/records/5519683	Four-month outdoor LoRa

link-quality dataset - PRIMARY CHOICE		connectivity/link-quality dataset. The record describes link-level SNR/RSS/PRR analysis, network-level correctly exchanged packets, node-level neighbour counts and temperature evolution. Strong fit for your feature engineering and validation story.
2. Emanuele Goldoni LoRa RSSI/SNR GitHub datasets	https://github.com/emanueleg/lora-rssi	Contains outdoor/indoor LoRa datasets. The vineyard dataset contains RSSI/SNR plus temperature, humidity and pressure from 8 LoRaWAN nodes and a reference weather station.
3. LoRa signal-quality time series - Zenodo	https://zenodo.org/records/13835721	CSV contains RSSI/SNR from three gateways, spreading factor and GPS/time information. Useful for spatial/temporal link-quality features.
4. LoRa SAR / avalanche datasets - Zenodo	https://zenodo.org/records/13932869	Real RSSI/SNR measurements collected in search-and-rescue scenarios at different snow/environment conditions. Useful as a second real-world environmental dataset.
5. LoRaWAN path-loss + environmental dataset - GitHub	https://github.com/magonzalezudem/MDPI_LoRaWAN_Dataset_With_Environmental_Variables	Large measurement campaign with 990,750 observations; includes distance, radio parameters, frame length, temperature, humidity, pressure, particulate matter, RSSI, SNR, time-on-air and energy.

Best starting point: use the ChirpBox dataset for link-quality/PRR concepts and the 990k-observation environmental dataset for RSSI/SNR/time-on-air relationships. Do not pretend either dataset is a direct multi-hop military mesh dataset; explicitly call them public LoRa measurement sources used to calibrate a prototype.

4. What to do when a dataset is missing labels

Your model needs a target. For the tomorrow demo, define the target as link_success = 1 if a packet is delivered within a chosen timeout and passes integrity checks; otherwise 0. If a downloaded public dataset does not expose row-level success, do not fabricate that label and call it ground truth. Instead, use the public data to build feature distributions and create a clearly labelled synthetic training dataset for the demo.

```
Example synthetic row

rssi=-74 dBm | snr=9.0 dB | pdr=0.98 | latency=95 ms | retries=0
battery=92%  | queue=12%  | toa=0.16 s                  -> success=1

rssi=-103 dBm | snr=-2.0 dB | pdr=0.55 | latency=650 ms | retries=5
battery=75%   | queue=72% | toa=0.80 s                       -> success=0
```

5. ML model: exact formulation

Train a supervised binary classifier per link observation. The classifier outputs the probability that a transmission will succeed. A second regression model can later predict latency; it is optional for tomorrow.

Input feature	Meaning	Priority tomorrow
RSSI	Received signal strength	High
SNR	Signal-to-noise ratio	High
PDR	Packet delivery ratio over a window	High
Latency	Measured or estimated round-trip/one-hop delay	High
Retries	Application-level retransmission count	High
ETX	Estimated transmission count; roughly inverse of delivery ratio	High
Queue %	Forwarding/processing backlog	Medium
Battery %	Node energy state	Medium
Temperature	Node operating state	Medium
Time-on-air	Radio airtime estimate	Medium
SF/BW/CR	LoRa radio configuration	Medium
Hop count	Path-level feature; keep separate from per-link model	Low/derived

```
Recommended feature vector
X = [RSSI, SNR, PDR, latency_ms, retries, ETX, queue_pct, battery_pct,
     temperature_c, time_on_air_s, spreading_factor, bandwidth]

Target
y = 1  -> packet successfully delivered within timeout and passes integrity check
y = 0  -> timeout, loss, or failed integrity check
```

Model choice: Random Forest is the safest first model. XGBoost is a strong second choice if you want a stronger tabular benchmark. Avoid an LSTM/Transformer for tomorrow unless you already have a large time-series pipeline.

6. How the route is calculated

Build a graph in which each wireless link is an edge. The ML model produces p_success for each edge. Convert that probability into a routing cost together with latency, retries and queue load. Then run a standard path algorithm such as Dijkstra.

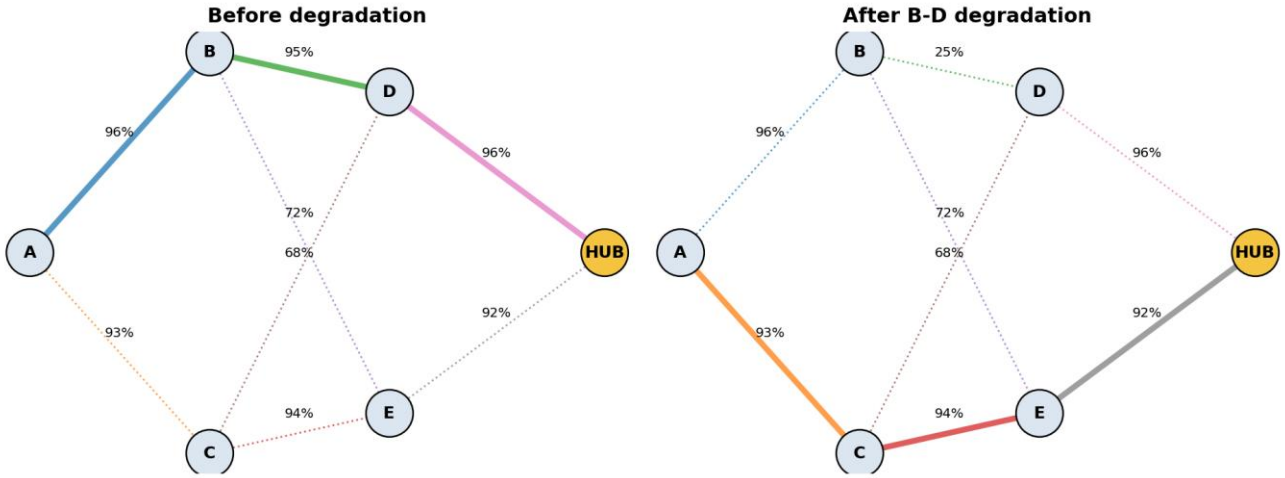
```
Example edge cost

edge_cost = -log(max(p_success, 1e-6))
            + 0.0012 * latency_ms
            + 0.05 * retries
            + 0.01 * queue_fraction
```

```
primary_path = shortest_path(source, hub, weight=edge_cost)
```

The important architectural separation is: ML predicts the condition of an edge; the routing algorithm chooses the path. This makes the system easy to debug and easy to explain to judges.

7. Live rerouting demo - exactly what the evaluator should see



1. Start the Streamlit dashboard with a healthy topology. The solid line is the primary route and the dashed line is the backup route.
2. Select one link in the sidebar, e.g. B -> D.
3. Lower SNR or RSSI, or lower PDR. Alternatively increase latency, retries or queue load.
4. The Random Forest probability for that edge decreases.
5. The routing graph recomputes edge costs.
6. When the degraded edge becomes expensive enough, the primary path switches to the alternate route.
7. The dashboard shows the old and new route, end-to-end reliability estimate and latency estimate.

Demo narrative: "At t0 the network prefers A -> B -> D -> HUB. I now degrade B -> D by reducing its SNR/PDR. The ML layer predicts a lower success probability, the graph assigns a higher cost, and the routing layer switches to A -> C -> E -> HUB. The nodes would then receive a route update in the physical implementation."

8. Software-only simulation options

Option	Use tomorrow?	What it gives you	Trade-off
Python + NetworkX + Streamlit	YES - recommended	Live topology, sliders, ML predictions, route changes	Not a full radio PHY simulation
FLoRa + OMNeT++	Later / optional	End-to-end LoRa simulation, nodes, gateways, network server, RSSI statistics, energy statistics	Setup/modeling overhead; not the fastest path for tomorrow
ns-3 + LoRaWAN module	Later	Large discrete-event simulator with packet tracking, topology output, energy integration	LoRaWAN-oriented rather than a ready-made LoRa mesh solution
Custom ns-3 LoRa mesh	Research phase	More realistic custom mesh model	High development effort

FLoRa facts checked online: FLoRa is an OMNeT++/INET framework for end-to-end LoRa simulation with nodes, gateways, a network server, ADR and energy statistics. The current project page lists a May 2026 release.

- FLoRa project: https://github.com/florasim/flora
- FLoRa documentation: https://flora.aalto.fi/howto/usage/
- ns-3 LoRaWAN module: https://github.com/signetlabdei/lorawan
- ns-3 LoRaWAN app page: https://apps.nsnam.org/app/lorawan/

9. Node-side architecture for the eventual hardware version

```
ESP32 + LoRa node

Radio driver
  -> packet manager
  -> ACK/timeout handler
  -> sequence number / duplicate detector
  -> neighbour table
  -> metric collector
  -> forwarding table
  -> telemetry sender

Node receives route update:
  destination = HUB
  primary_next_hop = B
  backup_next_hop = C
  route_version = 184
```

Node metric	How it is used
RSSI/SNR	Link-quality evidence
PDR / ACK success	Reliability evidence
Retries / ETX	Transmission cost
Latency / jitter	Performance cost
Queue depth	Avoid overloaded forwarding nodes
Battery	Avoid selecting a node that is near depletion
Temperature / heap / CPU	Node health; mainly a penalty or exclusion rule
Neighbour count	Topology and isolation indication

10. Message correctness - do not outsource this to ML

The ML model is not your integrity mechanism. A resilient communications prototype needs a deterministic packet protocol. Use a sequence number, integrity check, source/destination identifiers, and an end-to-end acknowledgement/timeout scheme. Duplicate packets can be dropped using the sequence number. This allows the demo to distinguish “radio link looked healthy” from “the application actually received the intended message.”

```
Packet header (conceptual)

version | source_id | destination_id | sequence | route_version | payload_len | integrity

Receiver:
1. check integrity
2. reject duplicate sequence
3. accept/forward
4. return ACK
```

11. What the live hub does

```
Every control interval

telemetry received
      |
      v
update node/link state
```

```
      |
      v
feature engineering
      |
      v
ML predicts p_success for each link
      |
      v
build weighted graph
      |
      v
compute primary + backup paths
      |
      v
if new path is materially better -> send route update
      |
      v
store result for later retraining
```

12. Route flapping control

Do not switch routes every time two scores differ by a tiny amount. Add hysteresis. For example, only switch if the candidate route is at least 5–10% better, or if the current route has stayed below a reliability threshold for several control intervals. This prevents unstable A/B/A/B route changes when a link is noisy.

13. Dataset pipeline for tomorrow

8. Download one real LoRa dataset and inspect its CSV columns.
9. Map its available fields to your common schema: timestamp, link/node identifier, RSSI, SNR, PDR/PRR if present, environmental values, radio configuration and any latency/airtime features.
10. Calculate derived values where possible, especially normalized RSSI/SNR and ETX from delivery ratio.
11. Create a separate synthetic data generator for missing features such as queue load, battery and CPU.
12. Label the synthetic training examples explicitly as SYNTHETIC-DEMO, not real measurements.
13. Train Random Forest and save the model with joblib.
14. Feed current edge metrics to the model from the Streamlit sidebar.
15. Convert probabilities into graph costs and recompute the route live.

```
Suggested project structure

mesh_ml_project/
  data/
    raw/
    processed/
    synthetic_demo.csv
  models/
    link_success_rf.joblib
  src/
    data_prep.py
    train_model.py
    routing.py
    simulator.py
  app.py
  requirements.txt
  README.md
```

14. Exact tomorrow roadmap

Time block	Task	Deliverable
0:00-0:45	Download + inspect data	CSV selected; feature mapping written
0:45-1:45	Synthetic augmentation	10k-50k labelled demo rows

1:45-2:45	Train Random Forest	Saved model + confusion matrix / ROC-AUC
2:45-4:15	Build routing graph	NetworkX primary + backup path
4:15-5:45	Build Streamlit demo	Topology + sliders + live route switching
5:45-6:30	Test scenarios	5 degradation scenarios + screenshots
6:30-7:15	Prepare explanation	Architecture, model, routing and limitations
7:15-8:00	Optional polish	Graphs, logging, model feature importance

Do not spend the critical time block on OMNeT++/FLoRa installation. Use those as your "future high-fidelity simulation" path. Your SIH proof-of-concept should be interactive and deterministic.

15. Five live test scenarios

Scenario	Change	Expected effect
1. Link fading	Lower RSSI and SNR on one edge	Probability falls; traffic moves to alternate path
2. Congestion	Increase queue % and latency	Path containing that node becomes more expensive
3. Retransmissions	Increase retry count / reduce PDR	ETX and link cost increase
4. Node health	Reduce battery and increase CPU/temperature penalty	Node can be penalized or excluded
5. Node failure	Set probability near zero or remove edge	Primary path disappears; backup path becomes active

16. Metrics to show on the final dashboard

- Current primary route and backup route
- Predicted p_success per edge
- Estimated end-to-end reliability of the selected route
- Estimated end-to-end latency
- Number of hops
- Packets sent / delivered / lost / retried (simulation counters)
- Route changes per minute (route stability)
- Node health summary: battery, queue, temperature, CPU/RAM when available

17. What you should and should not claim

Avoid saying	Say instead
"Uninterceptable communication"	"Resilient communication under degraded link conditions"
"Zero packet loss"	"Reduced packet loss through adaptive route selection and acknowledgements"
"AI finds the perfect route"	"ML estimates link quality and a routing algorithm selects the best available path"
"Real military performance validated"	"Prototype demonstrated with public LoRa measurements and controlled simulation"
"The model predicts exact field behaviour"	"The model is an experimental link-quality predictor and requires hardware validation"

18. Limitations you should state explicitly

- The Python dashboard is a logical network simulator, not a radio-physics emulator.

- LoRa is low-rate; ML cannot remove the physical airtime/throughput limit. The optimization is about choosing better links and avoiding unnecessary retransmissions.
- Public datasets vary in topology, hardware, environment and radio configuration; they are useful for feature realism but are not a substitute for your exact hardware.
- The synthetic dataset is suitable for demonstrating the pipeline but should never be reported as field validation.
- Eventually, hardware measurements must replace synthetic features such as battery, CPU and queue behaviour.

19. Implementation package included with this document

A ready-to-run starter demo is bundled separately with this document. It contains a Streamlit application that creates synthetic link data, trains a Random Forest, predicts edge success probabilities, computes primary/backup routes and lets you degrade a selected link live.

```
Run on Windows

cd mesh_ml_demo
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py

Then change a link's RSSI/SNR/PDR/latency/retry/queue values in the sidebar.
```

Important: the included model is a synthetic demonstration model. The first task after it works is to replace the synthetic training distribution with a public real LoRa dataset and clearly show that the demo model and the real-data model are different experiments.

20. References and links checked for this plan

- ChirpBox long-term LoRa link-quality dataset: https://zenodo.org/records/5519683
- LoRa RSSI/SNR experimental datasets: https://github.com/emanueleg/lora-rssi
- LoRa signal quality and GPS time-series dataset: https://zenodo.org/records/13835721
- LoRa SAR / avalanche dataset: https://zenodo.org/records/13932869
- LoRaWAN environmental/path-loss dataset: https://github.com/magonzalezudem/MDPI_LoRaWAN_Dataset_With_Environmental_Variables
- FLoRa: https://github.com/florasim/flora
- FLoRa documentation: https://flora.aalto.fi/howto/usage/
- ns-3 LoRaWAN module: https://github.com/signetlabdei/lorawan
- ns-3 LoRaWAN application page: https://apps.nsnam.org/app/lorawan/

Web information in this plan was checked on 18 August 2026. Dataset licensing and repository terms should be checked before redistribution.